Project Progress Report on

Voice-Based AI Assistant Using Local API Integration

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING CSE AI & DS

Submitted by:

Shashwat Malviya	2021746
Yash Rawat	2021768
Siddhant Srivastav	2021754
Retendra Negi	2021733

Under the Guidance of

Dr. Anita Saroj

Assistant Professor

Project Team ID: MP2025AI&DS5

Project Progress Report No: 1

Department of Computer Science and Engineering

Graphic Era (Deemed to be University)

Dehradun, Uttarakhand

2025-26

CANDIDATE'S DECLARATION

We hereby certify that the work which is being presented in the project progress report entitled **"Voice AI Assistant using Local API and Encryption "** in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering **B.Tech CSE AI & DS** in the Department of Computer Science and Engineering of the Graphic Era (Deemed to be University), Dehradun shall be carried out by the undersigned under the supervision of **Dr. Anita Saroj, Assistant Professor**, Department of Computer Science and Engineering, Graphic Era (Deemed to be University), Dehradun.

Shashwat Malviya	**2021746**
Yash Rawat	**2021768**
Siddhant Srivastava	**2021754**
Retendra Negi	**2021733**

The above-mentioned students shall be working under the supervision of the undersigned on
"Voice-Based AI Assistant Using Local API Integration"

Supervisor **Head of the Department**

<u>Examination</u>

Name of the Examiners **Signature with Date**
1.
2.

Table of Contents

Chapter No.	Description	Page No.
Chapter 1	Introduction and Problem Statement	
Chapter 2	Objectives	
Chapter 3	Project Work Carried Out	
Chapter 4	Future Work Plan	
Chapter 5	Weekly Task	
	References	

Chapter 1

Introduction & Problem Statement

Artificial Intelligence is rapidly transforming human–computer interaction. With the rise of voice-enabled assistants such as Siri, Alexa, and Google Assistant, interaction through natural speech has become increasingly popular. These assistants allow users to perform tasks such as searching the internet, opening applications, sending messages, or retrieving updates simply through spoken commands. However, most commercially available assistants rely heavily on cloud -dependent services, lack strong personalization, and do not offer robust security features for device-level usage—especially in shared computing environments like households, labs, or workplaces.

In addition, existing assistants follow restricted command patterns and cannot be easily customized for academic or personal use. They operate as black-box systems with no flexibility to integrate new modules, automation workflows, or personalized models. At the same time, advancements in Generative AI (such as transformer-based large language models) have made it possible for assistants to understand context, generate meaningful responses, summarize information, and perform higher-order reasoning. Yet, this capability has not commonly been merged with personalization and local processing.

A key challenge in creating a truly personalized voice assistant is ensuring secure access. Without authentication, anyone can interact with or misuse the assistant on a shared device. Biometric security, particularly face recognition, provides a non-intrusive and effective solution to ensure that only authorized users can access the assistant.

This project aims to design and develop a secure, intelligent, and customizable Voice AI Assistant that integrates:

Voice interaction, Generative AI reasoning, and Optional biometric authentication in a unified system.

The assistant captures voice commands, converts speech to text using STT models, processes the text using a transformer-based LLM, executes automation tasks via local system and Android ADB actions, and provides voice output through TTS models. An optional LBPH-based face recognition module enhances security and personalization.

Problem Statement

Existing voice assistants provide either voice interaction, automation, or AI-based responses, but they rarely combine secure authentication, privacy-preserving processing, contextual intelligence, and modular architecture within a single locally deployable system. Most commercial assistants rely heavily on cloud processing, which raises concerns regarding data privacy, storage of voice recordings, and potential misuse of transcripts. Additionally, traditional LLM-based systems often generate responses without grounding in specific domain knowledge, leading to hallucinations or inaccurate outputs. There is also limited focus on accessibility-driven enhancements and secure automation within desktop environments.

Therefore, the problem addressed in this project is the design and development of a secure, intelligent, and modular voice-based assistant that integrates biometric authentication, encrypted audio and transcript handling, Retrieval-Augmented Generation (RAG) for context-aware reasoning, and automation capabilities within a FastAPI-based local architecture. The system aims to ensure privacy, reduce hallucinations, enable reliable task execution, and provide a foundation for future accessibility-focused extensions.

Chapter 2

Objectives of the Project

The major objectives of the proposed work are as follows:

- To develop a secure authentication module using face recognition (LBPH/Embeddings) to ensure that only verified users can activate the assistant.

- To implement a Speech-to-Text (STT) module that converts user voice input into text using Whisper or Python SpeechRecognition.

- To integrate a Generative AI model (LLM) to analyze user queries, understand context, and generate human-like intelligent responses.

- To design a command execution and automation module that can perform tasks such as opening applications, launching websites, sending messages, or controlling Android devices using ADB.

- To implement a Text-to-Speech (TTS) module to convert AI responses or execution confirmations into spoken output using pyttsx3 or gTTS.

- To create a user-friendly frontend interface using Eel and web technologies (HTML/CSS/JS) for displaying transcribed queries, outputs, and assistant states.

- To develop a modular local API system using FastAPI for communication between voice, AI, automation, and UI modules.

Chapter 3

Work Completed in Phase-1

During Phase-1, the following work has been successfully completed:

- **Literature Survey**

A detailed review was conducted on:

Voice assistants and NLP-based systems

Whisper and SpeechRecognition STT models

Transformer architecture and LLMs

LBPH and CNN-based face recognition approaches

Eel-based desktop UI frameworks

Automation libraries such as PyAutoGUI and ADB

- **Speech-to-Text (STT) Module Implementation**

Audio is captured through the microphone.

Whisper and SpeechRecognition models tested for transcription.

The system converts user speech to text reliably in real-time.

Basic error handling and noise filtering implemented.

- **Generative AI Module Integration**

A preliminary connection to a GPT/HuggingFace LLM established.

User text queries are passed to the LLM through a local API endpoint.

Response returned and displayed in the UI.

Basic conversational interaction working.

- **Text-to-Speech (TTS) Output**

Implemented TTS using pyttsx3.

System can “speak” AI responses aloud.

Voice speed and pitch adjusted for naturalness.

- **Basic Frontend (UI) Setup Using Eel**

Created a simple UI screen with:

A “Speak” button

A transcript display area

An assistant reply area

Integrated UI with backend using Eel commands.

- **Local API Framework Setup**

Built initial FastAPI server with endpoints:

/stt

/llm

/tts

/execute

Tested all endpoints individually.

- **Optional Face Recognition — Data Collection Module**

Created scripts for capturing face images of the user. LBPH training script initialized.

Authentication module (verification) partially implemented and planned for Phase-2.

Chapter 4

Hardware and Software Requirements

- Hardware Requirements
- Processor: Intel i5 / AMD Ryzen 5 or higher
- RAM: Minimum 8 GB (recommended 16 GB)
- Storage: 256 GB SSD
- Webcam: For face recognition
- Microphone: For STT input
- Speakers/Headphones: For voice output
- Software Requirements

- OS: Windows 10/11 or Linux
- Programming Language: Python 3.10+
- Libraries:OpenCV,SpeechRecognition / Whisper,pyttsx3 / gTTS, HuggingFace Transformers / OpenAI, PyAutoGUI, FastAPI, Eel
- Database: SQLite
- Browser: Chrome/Edge for automation tasks

Chapter 5

Possible Approach / Algorithms Used

5.1 Possible Approach

The system follows a modular, secure, and extensible pipeline architecture where each component performs a specific function. The assistant integrates biometric authentication, encrypted audio handling, speech processing, Retrieval-Augmented Generation (RAG), automation capabilities, and user interaction through a web-based interface. The approach ensures privacy, modularity, and intelligent reasoning within a unified local API framework.

1. Optional Face Authentication

1. The system can begin by identifying the user through the webcam.
2. A facial image is captured and processed using the LBPH algorithm.
3. Trained face data is compared with the live input.
4. Only authenticated users are allowed to activate the assistant.
5. This step enhances personalization and provides a secure access layer.

2. Voice Capture and Encryption

1. After authentication, the user's voice is recorded through the microphone.
2. Audio preprocessing (noise reduction, normalization) is applied.
3. The recorded audio is encrypted using AES-based encryption before storage.
4. Plain audio exists only temporarily in memory.
5. Encrypted audio files prevent misuse of stored voice samples.

3. Speech-to-Text Conversion

1. Encrypted audio is temporarily decrypted in memory for processing.
2. The STT module converts speech into text using Whisper or SpeechRecognition.
3. Audio waveforms are transformed into spectrogram representations.
4. Neural acoustic models predict phonemes and words.
5. The generated transcript is passed to the next processing stage.
6. The transcript is encrypted before database storage

4. Intent Classification

1. The transcribed text is analyzed to determine user intent.
2. Keyword-based classification differentiates:
   - Command-based instructions
   - Informational queries
3. This enables routing of input to either automation or AI reasoning modules.
4. The system is designed for future semantic intent classification using embeddings or transformer models.

5. Retrieval-Augmented Generation (RAG) Integration

1. For informational queries, the system activates the retrieval module.
2. The query is converted into embeddings.
3. A vector database (FAISS/Chroma) performs similarity search.
4. Relevant document chunks are retrieved.
5. Retrieved context is injected into the LLM prompt.
6. The LLM generates grounded and context-aware responses.
   This approach reduces hallucination and improves response accuracy.

6. Generative AI Processing

1. The transformer-based LLM processes the augmented prompt.
2. Multi-head attention mechanisms analyze contextual relationships.
3. The model interprets semantic meaning and generates structured output.
4. Responses may include explanations, summaries, or conversational replies.
5. This forms the core reasoning engine of the assistant.

7. Command Mapping and Task Execution

1. If intent classification identifies a command, the automation module is triggered.
2. The system maps detected keywords to predefined actions.
3. Application-level tasks use OS commands or PyAutoGUI.
4. Web-based tasks use browser automation.
5. Mobile-based tasks may use ADB commands.
6. A confirmation response is generated after execution.

8. Text-to-Speech Output

1. The final text response is sent to the TTS module.
2. Libraries such as pyttsx3 or gTTS convert text into phonetic sequences.
3. Phonemes are synthesized into audio waveforms.
4. The output is played through speakers.
5. No plaintext audio responses are permanently stored.

9. Local API Communication and Security Layer

1. All modules communicate through a FastAPI-based local server.
2. The server runs on localhost (127.0.0.1).
3. API-key validation ensures authorized internal communication.
4. Separate endpoints exist for STT, RAG, LLM, automation, and TTS.
5. Encryption boundaries are maintained at storage layers.
6. This modular API design ensures scalability and secure integration.

5.2 Algorithms Used

A. LBPH for Face Recognition

1. Converts input face image into grayscale.
2. Extracts local binary patterns by comparing each pixel with neighbors.
3. Divides the image into regions and creates histograms for each.
4. Compares histograms of the input face with stored face templates.
5. Produces a confidence score for identity verification.

B. Whisper / SpeechRecognition for Speech-to-Text

1. Converts audio to a spectrogram representation.
2. Applies neural acoustic models to detect phonemes and words.
3. Uses language modeling to refine transcription accuracy.
4. Produces text output even under varying noise conditions.

C. Transformer-based LLM for AI Processing

1. Uses an encoder-decoder or decoder-only architecture.
2. Employs multi-head attention to analyze relationships between words.
3. Learns long-range dependencies and contextual meaning.
4. Generates coherent, human-like responses.

D. Keyword-based Command Mapping

1. Identifies specific keywords in the transcribed text.
2. Maps these keywords to predefined system actions.
3. Executes the task using scripts or automation tools.
4. Ensures fast processing for deterministic commands.

E. Text-to-Speech Conversion

1. Converts text into phonetic units.
2. Synthesizes phonemes into sound waves.
3. Outputs audio files or streams for playback.
4. Provides natural voice responses to the user.

F. Retrieval-Augmented Generation (RAG)

1. Converts user query into vector embeddings.
2. Performs similarity search in vector database.
3. Retrieves relevant document chunks.
4. Augments LLM prompt with retrieved context.
5. Generates grounded responses.
6. Reduces hallucination and improves factual accuracy.

5.3 Pseudocode of Full Flow

```
Code

Start
Initialize Local API Server
Optional: Perform Face Authentication

Capture Voice Input
Preprocess Audio
Encrypt Audio for Storage
Decrypt Audio in Memory

Convert Speech to Text (STT)
Encrypt Transcript for Storage

Classify Intent
If Command:
    Execute Automation
Else:
    Retrieve Relevant Context (RAG)
    Send Augmented Prompt to LLM

Generate Response
Convert Response to Speech (TTS)
Display Output on UI

Clear Sensitive Memory Buffers
End
```

Chapter 6

Future Work Plan

S No.	Task	Duration
1.	Complete face recognition authentication	1–2 weeks
2.	RAG Implementation	2 Weeks
3.	Add advanced AI features (summaries, email drafting)	1 week
4.	Improve UI with animations & user history	1 week
5.	Full automation integration (WhatsApp, YouTube)	2 weeks
6.	Optimize local API + logging	1 week
7.	Final testing & deployment	1 week
8.	Cloud ready workflow	1 week

Chapter 7

Weekly Task Report

Week	Completed Task
Week-1	Conducted an extensive literature survey covering existing voice assistants (Siri, Google Assistant, Alexa), Speech-to-Text systems, LLM-based conversational agents, and face recognition mechanisms. Studied relevant research papers, tool documentation (Whisper, Transformers, FastAPI), and finalized the problem statement and scope of the project.
Week-2	Implemented the Speech-to-Text (STT) pipeline using Whisper/SpeechRecognition. Set up microphone input capture, processed audio recordings, tested various STT models for accuracy, and integrated the transcription output with the backend. Completed basic error handling and noise-robustness checks.
Week-3	Developed the Generative AI module using the transformer-based model (GPT/HuggingFace). Integrated Local API endpoints (/llm) for text-based queries, built the request/response flow between backend modules, and validated the ability to generate contextual, natural responses. Performed initial prompt-engineering improvements.
Week-4	Implemented the Text-to-Speech (TTS) module and connected it with the UI. Developed an Eel-based frontend to display the transcript, AI output, and assistant states. Integrated backend responses into the UI workflow and ensured smooth real-time interaction across the pipeline.
Week-5	Performed full pipeline testing of STT → Query Routing → LLM Processing → Automation Module → TTS. Identified minor issues and refined the execution flow. Completed Phase-1 documentation including report writing, diagrams, weekly tracking, and preparation for evaluation.

Week-6	Focused on strengthening system security by designing encrypted audio storage and transcript protection. Updated the local API configuration to ensure secure, localhost-only communication and structured encryption boundaries.
Week-7	Initiated integration of Retrieval-Augmented Generation (RAG) using LangChain and a vector database for context-aware responses. Defined retriever workflow and created a dedicated API endpoint for RAG-based query handling.
Week-8	Refined the pipeline by separating command execution from LLM/RAG processing and improving intent classification logic. Began optimizing audio preprocessing and introduced confidence-based validation for safer execution.

References

1. OpenAI, Whisper: Robust Speech Recognition via Large-Scale Weak Supervision, Technical Documentation, 2022.
2. OpenCV Documentation, Face Recognition and Computer Vision Modules, OpenCV Official Guide.
3. A. Vaswani et al., “Attention Is All You Need,” Advances in Neural Information Processing Systems (NeurIPS), 2017.
4. HuggingFace, Transformers Library Documentation, 2023.
5. A. Zhang, SpeechRecognition Python Library Documentation, PyPI.
6. pyttsx3 Documentation, Text-to-Speech Conversion in Python, 2023.
7. S. Ramírez, FastAPI Documentation, 2018.
8. T. Ahonen, A. Hadid, and M. Pietikäinen, “Face Recognition with Local Binary Patterns,” European Conference on Computer Vision (ECCV), 2004.
9. P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems (NeurIPS), 2020.
10. LangChain Documentation, Building Applications with LLMs and Retrieval, 2023.
11. FAISS Documentation, Facebook AI Research, Similarity Search and Vector Clustering Library, 2023.
12. D. Boneh and V. Shoup, A Graduate Course in Applied Cryptography, Cambridge University Press (for AES and encryption principles reference).